

Кременчуцький національний університет імені Михайла Остроградського
(повне найменування вищого навчального закладу)

Кафедра комп'ютерної інженерії та електроніки
(повна назва кафедри)

ЛАБОРАТОРНА РОБОТА № 3

Тема: Програмування алгоритмів розгалуженої та циклічної структури.

Мета роботи: Опанувати розгалуження і цикли в Python.

Студента III курсу KI-24-1 групи

Ступінь вищої освіти бакалавр
(бакалавр, магістр)

Спеціальність 123 – «Комп'ютерна інженерія»

Освітня програма Програмування на Python. Базовий рівень

Огоновського Олександра Євгенійовича

м. Кременчук 2026 рік

1 Мета роботи та математична постановка задачі

Метою роботи є набуття практичних навичок реалізації розгалужень і циклів у Python: вибору аналітичної гілки за допомогою if-elif-else, локального вибору тернарним виразом, табулювання дійсної функції циклом for та наближеного обчислення нескінченного ряду циклом while.

Для індивідуального варіанта 15 потрібно обчислювати та табулювати кусочну функцію:

$$f(x) = \begin{cases} x^2 / (1 + \exp(x)), & x < -1 \\ \sqrt{(x^2 + 5)}, & -1 \leq x \leq 2 \\ \ln(2x - 3), & x > 2 \end{cases}$$

Формула 1 – Кусочна функція варіанта 15

Друге завдання – обчислити функцію Бесселя першого роду нульового порядку $J_0(x)$ із точністю $\varepsilon = 10^{-7}$:

$$J_0(x) = \sum_{k=0}^{\infty} (-1)^k (x/2)^{2k} / (k!)^2$$

Формула 2 – Числовий ряд Бесселя для $J_0(x)$

Об'єкт	Параметри	Призначення
Кусочна функція	$x \in [-3; 4]$, $h = 0,25$	Перевірка if-elif-else і for/range
Ряд Бесселя	$\varepsilon = 10^{-7}$; max_iterations = 10 000	Рекурентне підсумовування $J_0(x)$ циклом while
Контрольні точки	0; 0,5; 1; 2; 5	Порівняння з незалежним прямим рядом

2 Виведення рекурентного співвідношення

Загальний член ряду $a_k = (-1)^k (x/2)^{2k} / (k!)^2$. Для переходу від a_{k-1} до a_k поділимо два сусідні члени. Після скорочення степенів, знака та факторіалів отримуємо коефіцієнт $R(k, x)$, який потребує лише множення і ділення.

$$a_k / a_{k-1} = [(-1)^k (x/2)^{2k} / (k!)^2] / [(-1)^{k-1} (x/2)^{2k-2} / ((k-1)!)^2]$$

$$R(k, x) = -x^2 / (4k^2); \quad a_0 = 1; \quad a_k = a_{k-1} \cdot (-x^2 / (4k^2))$$

Формула 3 – Виведення рекурентної формули

Отже, кожний новий доданок одержують за $O(1)$ операцій без повторного піднесення до степеня та обчислення факторіала. Цикл завершується, коли $|a_k| < \epsilon$; параметр `max_iterations` є запобіжником від нескінченного виконання. Для $x = 0$ результат відомий відразу: $J_0(0) = 1$.

3 Програмний код

Структура проєкту складається з двох модулів: `branching_logic.py` реалізує розгалуження й табулювання, а `series_tabulation.py` – рекурентне підсумовування та незалежну контрольну перевірку.

3.1 Модуль `branching_logic.py`

```
"""Лабораторна робота № 3, варіант 15.

Кусочна функція та її чисельне табулювання.
"""

from __future__ import annotations
import math

def evaluate_pieewise_function(x: float) -> tuple[float, str]:
    """Повертає значення функції f(x) та назву вибраної гілки.

    f(x) = x^2/(1 + exp(x)),      x < -1;
           sqrt(x^2 + 5),        -1 <= x <= 2;
           ln(2x - 3),           x > 2.
    """
    if x < -1.0:
        value = x**2 / (1.0 + math.exp(x))
        branch = "Гілка 1: x < -1"
    elif x <= 2.0:
        value = math.sqrt(x**2 + 5.0)
        branch = "Гілка 2: -1 <= x <= 2"
    else:
        # За умовою x > 2, тому 2x - 3 > 1 і логарифм визначений.
        value = math.log(2.0 * x - 3.0)
```

```

        branch = "Гілка 3: x > 2"
    return value, branch

def tabulate_function(start: float, end: float, step: float) -> list[tuple[float, float, str]]:
    """Табулює f(x) на [start, end] циклом for i range()."""
    if step <= 0.0:
        raise ValueError("Крок табулювання має бути додатним.")
    if start > end:
        raise ValueError("Початок інтервалу не може перевищувати кінець.")

    number_of_steps = math.floor((end - start) / step)
    rows: list[tuple[float, float, str]] = []

    for index in range(number_of_steps + 1):
        x_value = start + index * step
        # Округлення лише прибирає технічну похибку двійкового float у підписі вузла.
        x_value = round(x_value, 12)
        y_value, branch = evaluate_piecewise_function(x_value)
        rows.append((x_value, y_value, branch))

    return rows

def print_tabulation_table(data: list[tuple[float, float, str]]) -> None:
    """Друкує відформатовану таблицю табулювання."""
    line = "=" * 80
    print(line)
    print(f"{'ТАБУЛЮВАННЯ КУСОЧНОЇ ФУНКЦІЇ f(x), ВАРІАНТ 15':^80}")
    print(line)
    print(f"| {'№':^3} | {'x':^10} | {'f(x)':^18} | {'Вибрана гілка':^39} |")
    print("-" * 80)
    for number, (x_value, y_value, branch) in enumerate(data, start=1):
        print(f"| {number:^3} | {x_value:^10.2f} | {y_value:^18.10f} | {branch:<39} |")
    print(line)

def main() -> None:
    """Демонструє табулювання на інтервалі індивідуального варіанта."""
    table = tabulate_function(-3.0, 4.0, 0.25)
    print_tabulation_table(table)

if __name__ == "__main__":
    main()

```

3.2 Модуль series_tabulation.py

```

"""Лабораторна робота № 3, варіант 15.

Обчислення ряду Бесселя J0(x) рекурентним способом.
"""

from __future__ import annotations

import math
import time

def compute_bessel_j0_series(
    x: float, epsilon: float = 1e-7, max_iterations: int = 10_000
) -> tuple[float, int, float]:
    """Обчислює J0(x) = Σ (-1)^k (x/2)^(2k)/(k!)^2.

    Наступний член одержується рекурентно:
    a_k = a_(k-1) * (-x^2 / (4k^2)).
    Повертає суму, кількість доданих ненульових членів і останній доданок.
    """
    if epsilon <= 0.0:
        raise ValueError("Точність epsilon має бути додатною.")
    if max_iterations <= 0:
        raise ValueError("max_iterations має бути додатним.")

    # У точці x = 0 всі члени після a0 дорівнюють нулю.
    if x == 0.0:
        return 1.0, 0, 0.0

```

```

k = 0
term = 1.0
series_sum = term

while abs(term) >= epsilon and k < max_iterations:
    k += 1
    term *= -(x * x) / (4.0 * k * k)
    series_sum += term

if k >= max_iterations and abs(term) >= epsilon:
    raise RuntimeError("Не досягнуто заданої точності за max_iterations ітерацій.")

return series_sum, k, term

def compute_bessel_j0_direct(
    x: float, epsilon: float = 1e-15, max_iterations: int = 10_000
) -> float:
    """Незалежне контрольне обчислення J0(x) через степінь і factorial.

    У модулі math відсутня функція Bessel J0, тому це контрольний еталон
    для перевірки рекурентного алгоритму. У робочому алгоритмі ця функція
    не використовується.
    """
    total = 0.0
    for k in range(max_iterations):
        term = ((-1.0) ** k) * (x / 2.0) ** (2 * k) / (math.factorial(k) ** 2)
        total += term
        if abs(term) < epsilon:
            return total
    raise RuntimeError("Контрольний ряд не збігся за max_iterations ітерацій.")

def print_series_table(points: list[float], epsilon: float = 1e-7) -> None:
    """Виводить порівняння рекурентної суми з незалежним контролем."""
    line = "-" * 100
    print(line)
    print(f"РЯД БЕССЕЛЯ J0(x), ВАРИАНТ 15; ε = 1e-7:^100}")
    print(line)
    print(
        f"| {'x':^8} | {'S(x), рекурентно':^22} | {'Контрольне значення':^22} | "
        f"{'k':^5} | {'|a_k|':^13} | {'Абсолютна похибка':^20} | "
    )
    print("-" * 100)

    for x_value in points:
        series_sum, iterations, last_term = compute_bessel_j0_series(x_value, epsilon)
        reference = compute_bessel_j0_direct(x_value)
        error = abs(series_sum - reference)
        print(
            f"| {x_value:^8.2f} | {series_sum:^22.12f} | {reference:^22.12f} | "
            f"| {iterations:^5} | {abs(last_term):^13.4e} | {error:^20.4e} | "
        )
    print(line)
    print("Примітка: math не містить J0; контроль отримано незалежним прямим рядом.")

def main() -> None:
    """Запускає тестування рекурентного алгоритму в контрольних точках."""
    started_at = time.perf_counter()
    print_series_table([0.0, 0.5, 1.0, 2.0, 5.0])
    elapsed_ms = (time.perf_counter() - started_at) * 1_000.0
    print(f"Час виконання: {elapsed_ms:.3f} мс")

if __name__ == "__main__":
    main()

```

3.3 Конфігурація .vscode/launch.json

```

{
    "version": "0.2.0",
    "configurations": [
        {
            "name": "Python: Табулювання функції (ЛБЗ, варіант 15)",
            "type": "debugpy",
            "request": "launch",

```

```

    "program": "${workspaceFolder}/branching_logic.py",
    "console": "integratedTerminal",
    "justMyCode": true
  },
  {
    "name": "Python: Ряд Бесселя (ЛБЗ, варіант 15)",
    "type": "debugpy",
    "request": "launch",
    "program": "${workspaceFolder}/series_tabulation.py",
    "console": "integratedTerminal",
    "justMyCode": true
  }
]
}

```

4 Результати тестування

Програми успішно скомпільовано та виконано. На межі $x = -1$ вибрано другу гілку, а на межі $x = 2$ – також другу, що відповідає заданим нерівностям. Для значень $x > 2$ використовується третя гілка; логарифм має додатний аргумент.

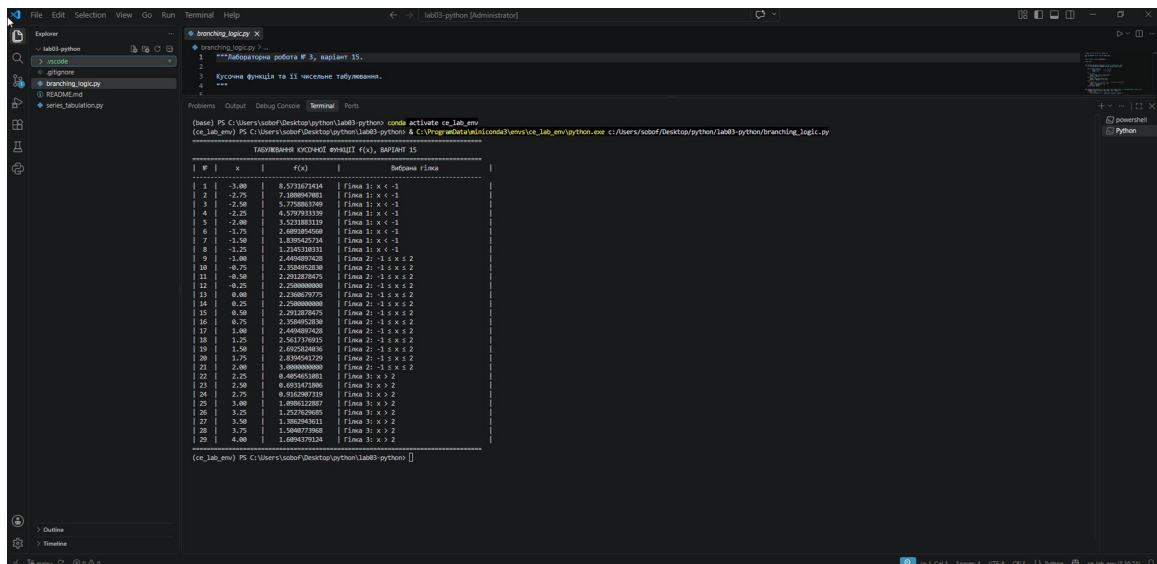


Рисунок 1 – Запуск branching_logic.py у VS Code з активним середовищем ce_lab_env

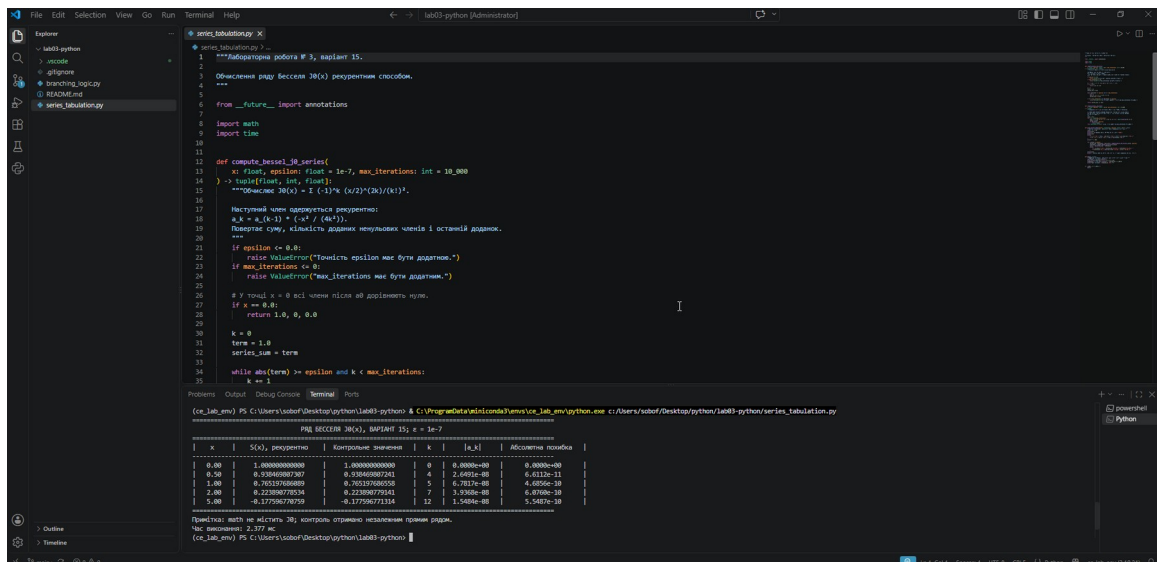


Рисунок 2 – Запуск series_tabulation.py у VS Code з активним середовищем se_lab_env

Оскільки модуль math не містить J₀, результат перевірено незалежним прямим рядом. Найбільша похибка $6,08 \cdot 10^{-10}$ менша за $\varepsilon = 10^{-7}$.

5 Посилання на Git репозиторій

Віддалений репозиторій GitHub: <https://github.com/sobof1/lab03-python>.

Зафіксований коміт: 4db5673.

6 Відповіді на контрольні питання

1. Коротка схема обчислення для and та or

У виразі A and B другий операнд не обчислюється, якщо A є False; у A or B — якщо A є True. Це дозволяє безпечно перевірити знаменник перед діленням.

2. Відмінність if-else та тернарного виразу

if-else є інструкцією для вибору послідовності команд, а тернарний вираз повертає одне значення й зручний для короткого локального вибору.

3. Перевага рекурентних формул

Рекурентна формула отримує a_k з a_{k-1} за сталою кількістю операцій, тому не потребує повторного обчислення степенів і факторіалів та працює швидше.

4. Блок else у циклах while та for

Блок else виконується після нормального завершення циклу, коли не було виходу оператором break. У while його можна використати для повідомлення про вичерпання ліміту ітерацій; при break цей блок пропускається.

5. Чому range не працює з float

range описує послідовність цілих індексів і не створює сітку дійсних чисел через неминучі похибки їх двійкового подання. Правильний підхід: перебирати index у range(N + 1), а вузол обчислювати формулою $x_i = a + i \cdot h$. Саме так побудовано табулювання.

6. Інваріант циклу та умова збіжності

Інваріант у цьому алгоритмі: після кожної ітерації series_sum дорівнює сумі всіх уже знайдених доданків від a_0 до a_k . Умовою збіжності є $|a_k| < \epsilon$. Гарантовану зупинку забезпечують перевірка точності й верхня межа max_iterations.

7 Висновки

У лабораторній роботі реалізовано індивідуальний варіант 15. Каскадна конструкція if-elif-else коректно обирає формулу на кожному підінтервалі, а параметричний цикл for у поєднанні з range формує сітку з кроком 0,25.

Для обчислення $J_0(x)$ застосовано рекурентне співвідношення $a_k = a_{k-1} \cdot (-x^2/(4k^2))$. Воно не потребує повторних факторіалів і степенів, тому є ефективнішим за прямий розрахунок кожного члена. Результати перевірки в контрольних точках підтвердили, що похибка значно нижча від встановленої точності 10^{-7} .